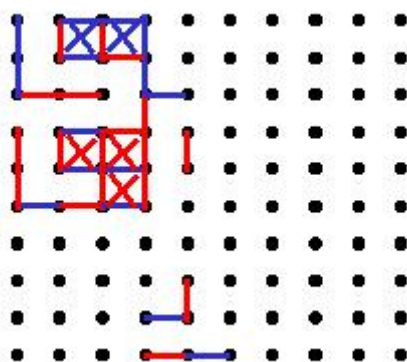


Trabalho Prático 1



Infraestruturas Computacionais Distribuídas

Instituto Superior de Engenharia de Lisboa 2025/2026

Licenciatura em Engenharia Informática e Multimédia

Docente responsável pela UC: Porfírio Filipe

Professor: Porfírio Filipe (turma 41D)

26/04/2026

ALUNO: Rui Ferreira / 51597 / A51597@alunos.isel.pt

Índice

Índice	2
Introdução	3
Arquitetura do sistema	3
Server Layer (Servidor Concorrente)	3
Client Layer (Cliente Gordo / Fat Client)	4
Shared Layer (Common)	5
Camada de transporte e portas de rede	6
Persistência de dados	7
Protocolo	7
Demonstração do funcionamento	8
Conclusão	8
Anexos:	11
Anexo 1 - Componentes do Cliente:	11
Anexo 2 - Dependências do Cliente:	11
Anexo 3 - Componentes da Common package:	12
Anexo 4 - Dependências da Common package:	13
Anexo 5 - Componentes do Servidor:	14
Anexo 6 - Dependências do Servidor:	15
Anexo 7 - Exemplo de armazenamento por XML:	15
Anexo 8 - Demonstração de Login:	16
Anexo 9 - Demonstração de ver perfil:	16
Anexo 10 - Demonstração de procurar por Players:	17
Anexo 11 - Demonstração do ecrã de jogo e de realizar uma jogada:	17
Anexo 12 - Demonstração da opção de desistir:	18

Introdução

Este relatório apresenta a arquitetura e os detalhes de implementação do primeiro trabalho prático da unidade curricular de Infraestruturas Computacionais Distribuídas (IECD). O objetivo principal deste projeto consistiu na conceção e implementação de uma infraestrutura computacional distribuída que permite a utilizadores registados jogar o jogo de estratégia por turnos "Pontos e Caixas" (*Dots and Boxes*) numa rede de computadores.

A solução desenvolvida é uma aplicação robusta com arquitetura Cliente-Servidor escrita em Java 25, estruturada como um único módulo Maven. Apresenta uma interface de linha de comandos (CLI) baseada no terminal que permite aos utilizadores efetuarem o registo e autenticação de forma segura, gerirem os seus perfis, procurarem por outros jogadores, enviarem convites para jogar e disputarem partidas em tempo real.

Arquitetura do sistema

A solução desenvolvida assenta numa arquitetura clássica Cliente-Servidor, centralizada num único módulo Maven e organizada internamente em três pacotes principais: `client`, `server` e `common`. Esta divisão garante um forte desacoplamento entre a interface do utilizador, a “business logic” e as regras intrínsecas do jogo.

(Os diagramas UML do projeto podem ser encontrados nos anexos 1 a 6)

Abaixo detalham-se as competências e a natureza de cada layer:

Server Layer (Servidor Concorrente)

O Servidor atua como a entidade central e autoritária do sistema. A sua execução baseia-se num ponto de entrada Singleton (`iecd.a51597.server.Server`). Este nó é estruturalmente concorrente, permitindo que múltiplos clientes interajam em simultâneo:

- **Gestão de Rede:** A classe “`ListenerThread`” escuta de forma contínua a porta de rede configurada em “`ServerConfiguration.DEFAULT_PORT`” e, por cada nova ligação de um cliente, instância uma *thread* dedicada gerida pela classe “`Connection`”.

- **Encaminhamento de Mensagens:** O tráfego que chega é extraído dos *frames* TCP e interpretado pela classe “MessageDispatcher”. Esta entidade funciona como o *router* central do servidor, extraíndo o tipo de ação (“ActionType”) da mensagem recebida e encaminhando o pedido para o seu *handler* dedicado, como o “AuthHandler”, “ProfileHandler” ou “GameHandler”. O message dispatcher executa a desserialização da mensagem através de uma implementação da classe “CommParser” (neste caso, XMLParser), que funciona como *translation layer* da mensagem que o servidor recebe, para a classe própria “Message”.
- **Controle de Estado:** A veracidade do sistema é assegurada por gestores em memória. O “SessionManager” valida e controla os tokens de utilizadores ativos na rede, enquanto o “GameManager” rastreia os convites pendentes e orquestra o estado das partidas a decorrer. Os vários utilizadores do sistema são desserializados no *startup* do servidor pela classe “PersistenceManager”, que age como wrapper à volta de uma classe que implemente “UserRepository” (que atualmente é “XMLUserRepository”, mas que pode ser facilmente trocada para outra implementação que use outros recursos, como uma base de dados SQL).
- **Implementação do jogo:** A implementação do jogo é feita de forma completamente modular, tornando o servidor num sistema capaz de suportar vários tipos de jogos diferentes através do uso de interfaces. Cada módulo de jogo deve implementar as seguintes interfaces:
 - **Game**, responsável pela lógica do jogo;
 - **GameFactory**, responsável por associar a cada jogo um “MoveCodec” dedicado, e a criação de partidas novas;
 - **Move**, responsável por representar o turno de um jogador no jogo;
 - **MoveCodec**, responsável pela serialização e desserialização das mensagens cliente-servidor e vice-versa.
 - **MoveResult**, responsável por representar o resultado de uma “Move” no jogo.

Client Layer (Cliente Gordo / Fat Client)

A *Client Layer* gere o estado da sessão local do utilizador e encarrega-se da Interface de Linha de Comandos (CLI). Na presente arquitetura, ele enquadra-se na

tipologia de Cliente Gordo (Fat Client) devido às pesadas responsabilidades que assume quer na navegação, quer no processamento local do estado do jogo:

Interface e Navegação UI: O ciclo de leitura do terminal (ClientCliHandler) interage com uma StateMachine baseada num stack de classes “Screen”. Esta máquina de estados dita a transição fluida entre diversos ecrãs, encapsulando as screens como “LoginScreen”, “MainMenuScreen” ou “GameScreen”. Para fácil construção de screens novas, é também disponibilizada a classe “OptionsScreen”, uma classe abstrata que herda de “Screen” e implementa sobre esta o armazenamento de *records* “ScreenOption”.

Lógica de Jogo Local: Através do componente “GameController”, o cliente mantém em memória uma cópia local do estado do tabuleiro (DotsAndBoxesGame). O cliente verifica localmente a validade de qualquer *Move* antes que esta seja enviada para o servidor, onde é verificada novamente. Esta abordagem fornece feedback visual imediato, embora o Servidor mantenha sempre a validação final autoritária. Adicionalmente, a complexidade de desenhar o tabuleiro na consola está inteiramente delegada ao cliente, através do “ClientBoardRenderer”.

É importante mencionar que embora o servidor apresente a capacidade de facilmente adicionar novos *game modules* através das interfaces criadas, o cliente não dispõe da mesma funcionalidade, sendo completamente dependente do jogo que implementa. Esta escolha de design foi feita em parte devido à complexidade do desenvolvimento de um cliente que permita módulos de jogo, e em parte pela escassez de tempo disponível para a realização do trabalho.

Shared Layer (Common)

Embora não seja um “nó” de execução autónoma, o pacote `iecd.a51597.common` é a espinha dorsal de comunicação do sistema. Este contém as definições do protocolo de mensagens (neste caso, XML), os Data Transfer Objects (DTOs) e as abstrações fundamentais do motor de jogo (Game, MoveCodec, etc...). Sendo a implementação do jogo (Dots and Boxes) alojada neste módulo comum, garante-se de forma elegante que o Cliente e o Servidor aplicam, cada um na sua esfera, o mesmo conjunto de regras ao longo de toda a sessão.

Camada de transporte e portas de rede

Para suportar a comunicação entre os vários nós da infraestrutura, a arquitetura foi desenhada sobre o protocolo de transporte **TCP** (Transmission Control Protocol) através da utilização de *Sockets*. A escolha do TCP, em detrimento do UDP, justifica-se pela necessidade absoluta de fiabilidade na entrega das mensagens sobre a velocidade destas: num jogo de estratégia por turnos como o Dots and Boxes, a perda, duplicação ou entrega desordenada de uma jogada ou de um pacote de autenticação corromperia irremediavelmente o estado do jogo e a sincronização entre o Cliente e o Servidor.

Uma vez que o TCP é um protocolo “stream-oriented” e não preserva as fronteiras das mensagens nativamente, foi implementada uma estratégia de enquadramento de dados (framing). Cada mensagem enviada para a rede é precedida por um prefixo de 4 bytes (um Big-Endian Integer) que indica o tamanho exato do *frame* (o documento XML transmitido, no caso atual) que se segue. A leitura destes fluxos é gerida pela classe “Connection” no Servidor e pela classe “ServerConnection” no Cliente.

No que respeita às portas de rede, o sistema foi concebido para ser totalmente configurável, não possuindo valores fixos em código (hardcoded). A configuração é efetuada no momento do arranque das layers através da leitura de ficheiros XML:

- O servidor determina a porta onde ficará à escuta (por exemplo, o porto 5555, o default) lendo o ficheiro “config/config.xml”, cujo processamento fica a cargo da classe ServerConfiguration.
- O cliente, por sua vez, identifica o endereço IP e o porto de destino do Servidor consultando o ficheiro “config/client_config.xml”, gerido pela classe ClientConfiguration.

Ambas estas classes disponibilizam as variáveis configuradas definindo-as como variáveis “static”.

Esta abordagem garante flexibilidade na implantação da infraestrutura em diferentes ambientes de rede sem necessidade de recompilar o código-fonte, ou utilizar variáveis de ambiente, permitindo que a sua implementação num novo sistema seja *plug-and-play*.

Persistência de dados

O sistema incorpora um mecanismo para manter o estado dos jogadores de forma persistente entre execuções do servidor. Os dados a armazenar incluem as credenciais de autenticação, o perfil do jogador (alunha, fotografia, nacionalidade e idade) e as suas estatísticas de jogo, encapsuladas na classe `PlayerStats` (que regista o número de vitórias, derrotas e o tempo total de jogo).

Durante a execução normal do servidor, o estado ativo do sistema é mantido em memória RAM pela classe `UserStore`, que utiliza mapas concorrentes para indexar os utilizadores de forma eficiente (por UUID e por Username). Para garantir que estes dados não se perdem quando o servidor é encerrado, foi desenvolvida a classe `PersistenceManager`. Este componente lança uma thread em segundo plano (background thread) que efetua periodicamente a gravação de todo o estado do `UserStore` para o disco (Que executa de 1 em 1 minuto por default, mas que permite a sua configuração através do ficheiro XML de configuração do servidor).

A persistência física dos dados é delegada ao padrão `Repository`, mais especificamente à implementação `XmlUserRepository`. As principais características desta solução são:

- **Formato de Armazenamento:** Os dados são guardados num ficheiro de texto no formato XML, tipicamente localizado em `data/users.xml` no diretório de trabalho do servidor (sendo esta pasta excluída do controlo de versões por questões de privacidade e segurança dos dados locais) (ver [anexo 7](#) para exemplo de armazenamento no ficheiro XML).
- **Estrutura e Validação (XSD):** A estrutura do ficheiro XML é estritamente definida e validada contra o esquema `schemas/users.xsd`.

Como já referido, a persistência de dados é também feita de forma modular, permitindo a fácil troca do módulo `XmlUserRepository`, por outro módulo que realize o armazenamento dos dados do servidor de outra forma, como um hipotético `SqlUserRepository`, ou um `JsonUserRepository`.

Protocolo

(Dada a riqueza e complexidade do protocolo desenvolvido, recomendo vivamente a consulta do documento `docs/protocol-reference.md` disponível nos ficheiros do

projeto, onde se detalha de forma exaustiva a sintaxe e semântica de todo o protocolo de comunicação.)

Para suportar as diversas interações exigidas pelo sistema (autenticação, convites, jogadas), foi concebido um protocolo de aplicação assente na troca de mensagens no formato XML.

A arquitetura deste protocolo destaca-se pelos seguintes pilares:

- **Sintaxe e Validação (XSD):** Todas as mensagens em trânsito obedecem a uma estrutura padronizada, composta por metadados descritivos e um corpo (MessageBody). Através da classe central “XMLParser”, qualquer frame recebido é estritamente validado contra o esquema “schemas/protocol.xsd” antes de ser processado.
- **Semântica e Ações:** A intenção de cada mensagem é governada por duas enumerações chave: “MessageType” (REQUEST, RESPONSE, PUSH) e o ActionType (que define a operação concreta, por exemplo, LOGIN, GAME_INVITE ou GAME_MOVE).
- **Construção Abstrata:** O código não manipula strings XML de forma manual. A serialização de objetos Java para os pacotes de rede é efetuada de forma segura com recurso ao padrão Builder, concretamente através das classes “XMLClientMessageBuilder” e “XMLServerMessageBuilder” que transformam os documentos XML em instâncias da classe “Message”.
- **Tratamento de Erros (Fail-Fast):** Qualquer anomalia na estrutura da mensagem (ataques ou falhas de software) gera instantaneamente uma “MalformedMessageException”. Estas exceções são mapeadas num “ErrorCodeType”, garantindo que o ciclo é interrompido com segurança e que o cliente recebe um código de erro inequívoco para apresentar na interface.

Demonstração do funcionamento

(Anexos 8 a 12)

Conclusão

O desenvolvimento deste projeto permitiu consolidar com sucesso os conceitos práticos de infraestruturas distribuídas, culminando num sistema Cliente-Servidor funcional para o jogo Dots and Boxes. A separação rigorosa entre a lógica de

interface, o motor de regras e a camada de transporte resultou num software modular e robusto.

Em resposta ao requisito final do guião, apresenta-se a avaliação crítica da arquitetura proposta:

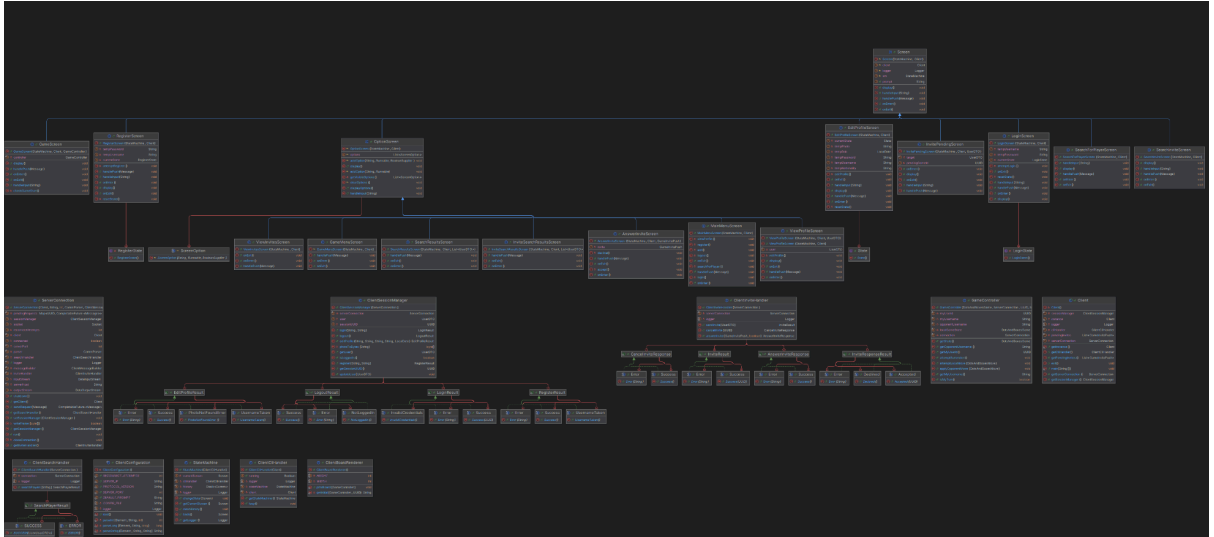
- **Expansibilidade (Vantagem):** O sistema apresenta uma elevada expansibilidade a vários níveis. No domínio da jogabilidade, a abstração do motor de jogo no pacote “common” através das interfaces “Game” e “MoveCodec”, juntamente com o padrão “GameFactory”, permite que a introdução de um novo jogo de tabuleiro exija apenas a criação de novas implementações destas classes, sem necessidade de alterar o protocolo de rede ou a máquina de estados. Adicionalmente, ao nível do armazenamento, o sistema utiliza o padrão Repository: a dependência da interface “UserRepository” garante que a atual persistência em XML (“XmlUserRepository”) pode ser facilmente substituída no futuro por outros mecanismos (como uma base de dados SQL ou MongoDB), bastando criar uma nova implementação desta interface sem qualquer impacto na lógica de negócio do Servidor.
- **Concorrência (Vantagem):** O Servidor está perfeitamente apto a lidar com múltiplos jogadores em simultâneo. A arquitetura delega cada ligação para uma thread independente gerida pela classe “Connection”. Adicionalmente, o estado partilhado (como o UserStore) recorre a mapas concorrentes em memória para garantir operações seguras e mitigar condições de corrida (race conditions).
- **Tolerância a Falhas (Vantagem/Desvantagem):** Ao nível da rede, o uso de TCP com framing garante a entrega ordenada e fiável dos pacotes XML. Caso ocorram quebras abruptas de ligação (falhas de rede ou encerramento forçado do cliente), o servidor deteta essas anomalias levantando uma “CommException”, o que permite ao SessionManager limpar o estado atempadamente e informar o adversário. A principal desvantagem da solução atual reside na persistência periódica: como o PersistenceManager guarda os dados em disco apenas em intervalos regulares, uma falha catastrófica de energia no servidor (antes do ciclo de gravação) pode resultar na perda das estatísticas dos últimos momentos.
- **Segurança (Vantagem):** O sistema aplica uma filosofia *fail-fast*: o “XMLParser” valida estritamente todas as mensagens contra os esquemas XSD (protocol.xsd). Qualquer tentativa de injeção de XML malformato por clientes gera imediatamente uma “MalformedMessageException”, tornando o servidor resistente contra ataques e corrupção de estado.

- **Transparência (Vantagem):** A infraestrutura oferece total transparência ao utilizador final. O jogador interage com uma Interface de Linha de Comandos através da “StateMachine” do Cliente. Toda a complexidade da serialização XML, validação de esquemas, gestão de Sockets TCP e resolução de Little/Big-Endian está completamente encapsulada e invisível na camada de apresentação.

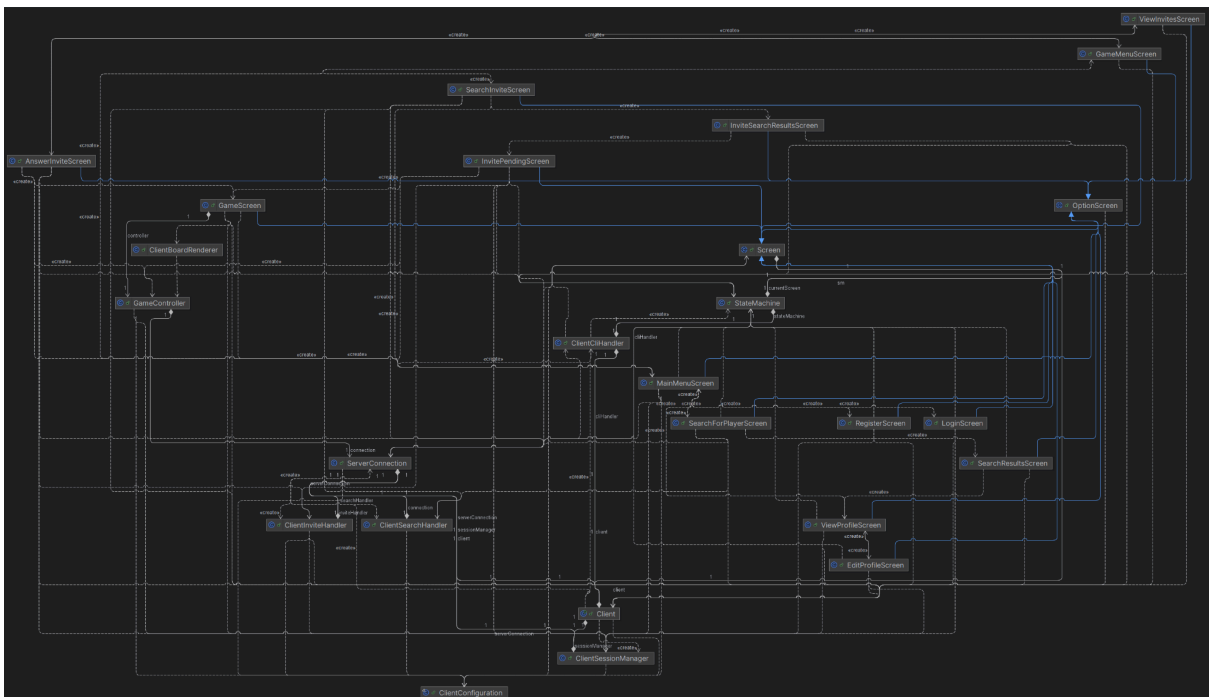
Em suma, a solução cumpre integralmente os requisitos propostos, entregando um ecossistema de jogo em rede seguro, escalável e de fácil manutenção arquitetural.

Anexos:

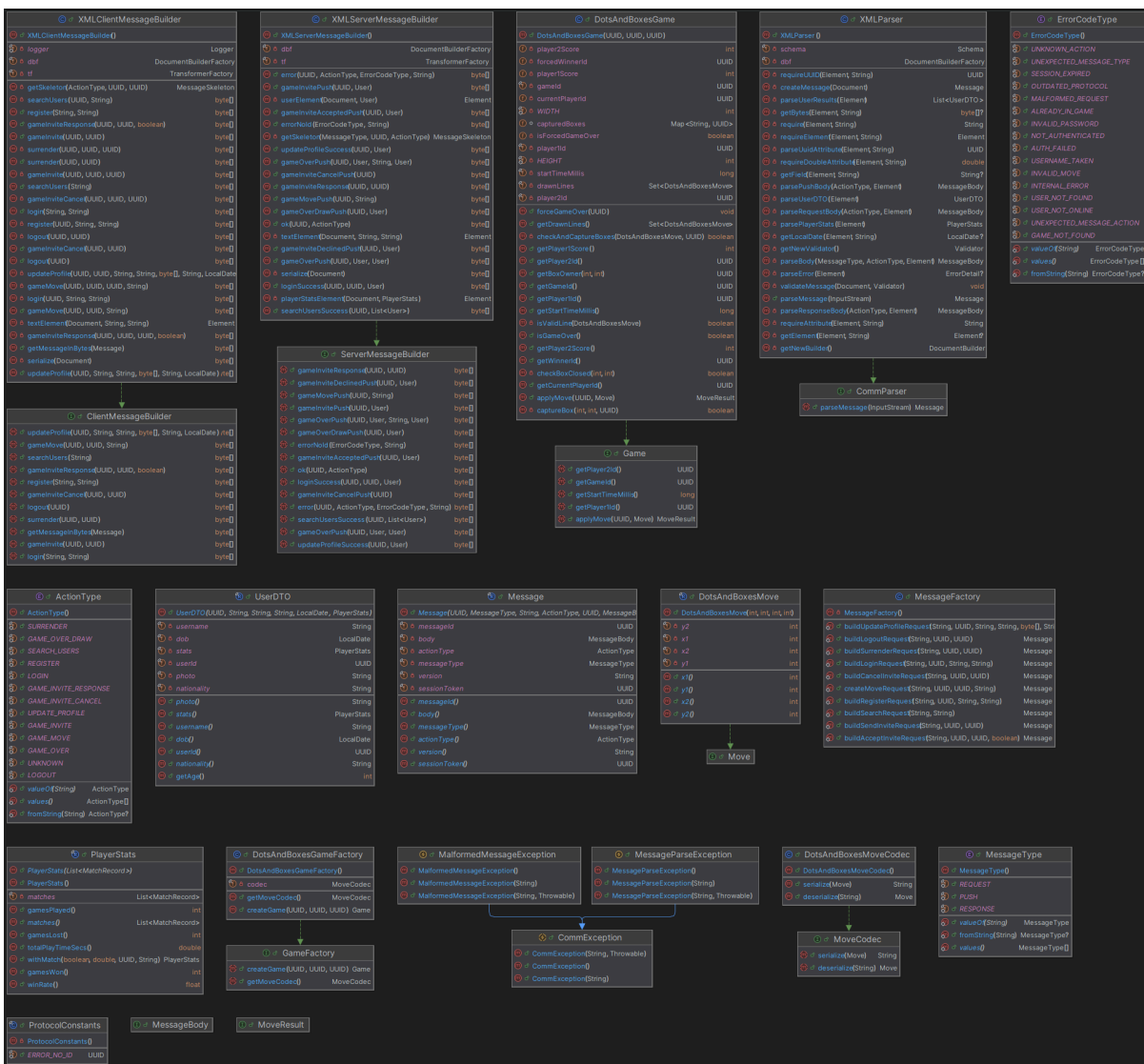
Anexo 1 - Componentes do Cliente:



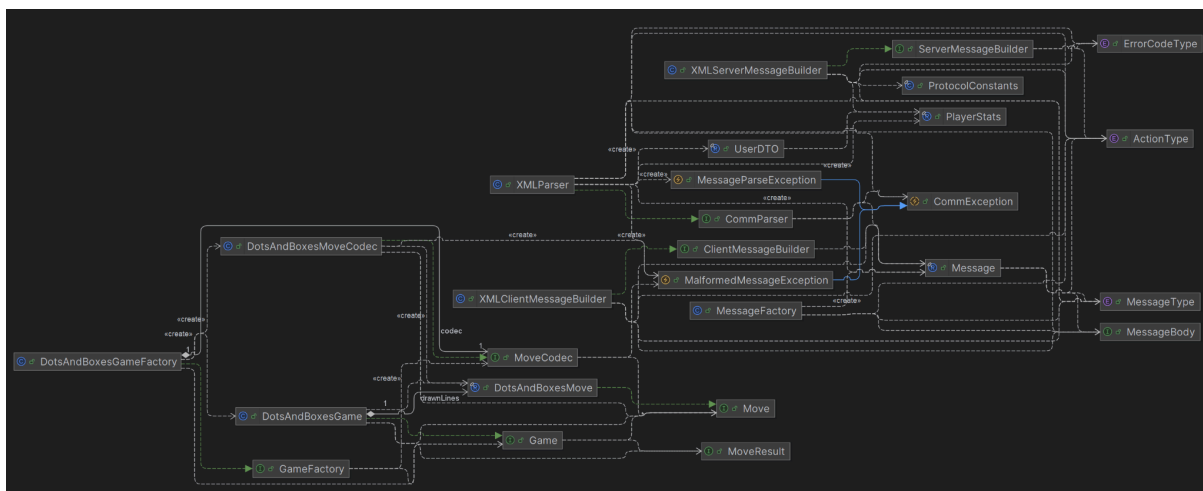
Anexo 2 - Dependências do Cliente:



Anexo 3 - Componentes da Common package:



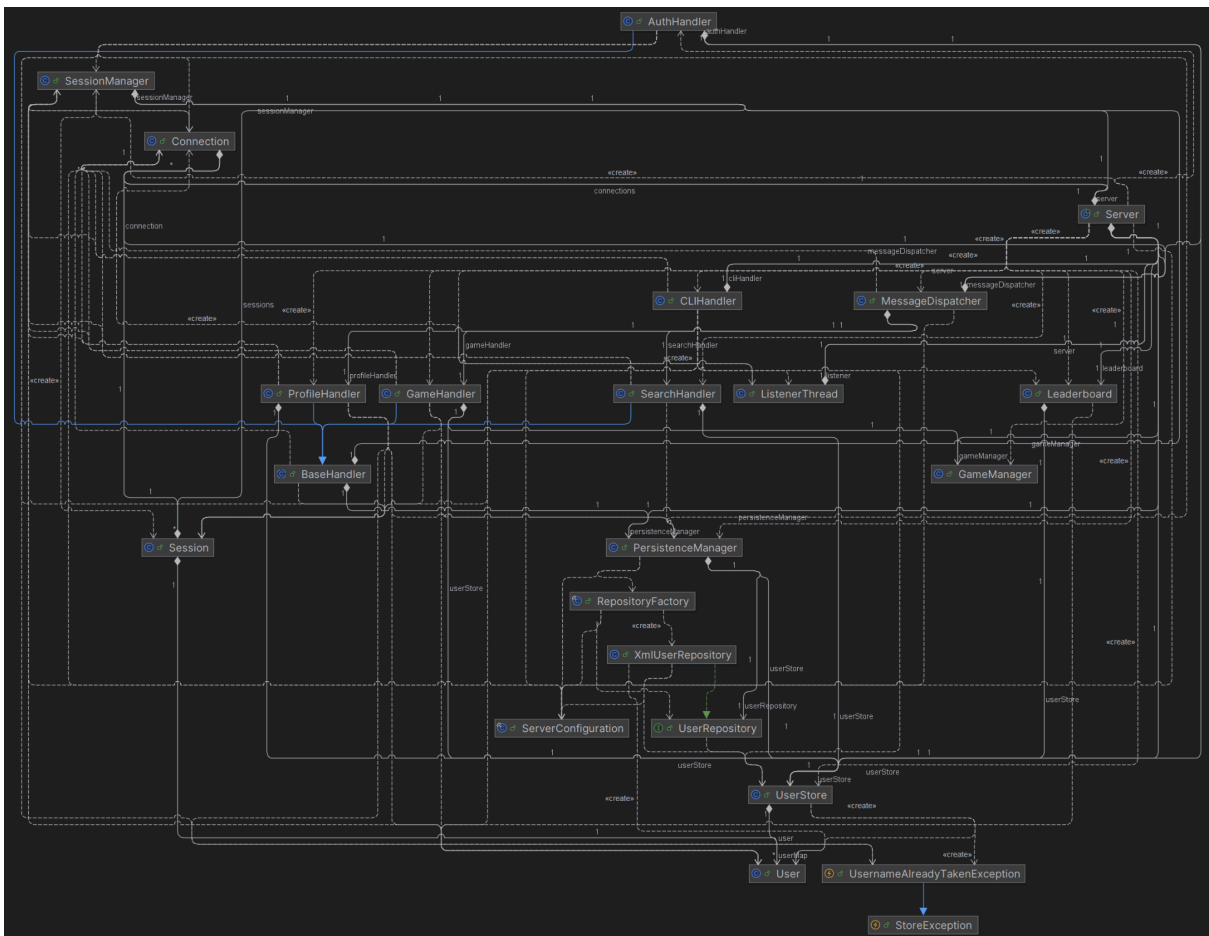
Anexo 4 - Dependências da Common package:



Anexo 5 - Componentes do Servidor:



Anexo 6 - Dependências do Servidor:



Anexo 7 - Exemplo de armazenamento por XML:

```
<?xml version="1.0" encoding="UTF-8"?><users>
  <user dob="2005-04-11" id="1b017df8-36d8-4dc4-a39d-e0b8778ca6c8" nationality="pt" passwordHash="$2a$12$HWVzEzpQ4V//56w3SE0Wc0gezwpC39LngI2c7EMdFwyJq074ULqe" photo="636118cc-3267-4624-af44-a8fa63ee8379.png"
  username="test">
    <stats>
      <match opponent-id="6151575b-8ecf-4eac-b760-fd28a3d2c8d0" opponent-username="ru1" playtime="32.624" result="LOST"/>
      <match opponent-id="6151575b-8ecf-4eac-b760-fd28a3d2c8d0" opponent-username="ru1" playtime="153.894" result="LOST"/>
      <match opponent-id="6151575b-8ecf-4eac-b760-fd28a3d2c8d0" opponent-username="ru1" playtime="20.316" result="WON"/>
    </stats>
  </user>
  <user dob="2005-04-11" id="bee3864f-4c4d-4736-8f60-6c01c65f976c" nationality="pt" passwordHash="$2a$12$LCG3rQ.Mbr5Vf0sFER8300t.Ly66aPGUbuF66I.PLMLPLMLVkdq" photo="33796c9c-65e5-41e5-b56e-d2ad13312484.png"
  username="test2">
    <stats/>
  </user>
  <user id="6151575b-8ecf-4eac-b760-fd28a3d2c8d0" passwordHash="$2a$12$8eyfu/6.MFilyxv1T0Kex.H9zNlKXcYV61oGPYpovbp2T9q0csdsW" photo="a6c6d593-aa4f-432d-9a56-df71bcc313f6.png" username="ru1">
    <stats>
      <match opponent-id="1b017df8-36d8-4dc4-a39d-e0b8778ca6c8" opponent-username="test" playtime="32.624" result="WON"/>
      <match opponent-id="1b017df8-36d8-4dc4-a39d-e0b8778ca6c8" opponent-username="test" playtime="153.894" result="WON"/>
      <match opponent-id="1b017df8-36d8-4dc4-a39d-e0b8778ca6c8" opponent-username="test" playtime="20.316" result="LOST"/>
    </stats>
  </user>
  <user dob="2000-05-12" id="13559c58-c53f-4f0e-915e-f03d302b6dab" nationality="en" passwordHash="$2a$12$07WVz20HFSjt.Ck99n3p110FHKfH6c94oKSMVoEyy8.hrQ6UkbtDsa" photo="c0dfbf98-adf5-441e-94c4-9faec23fa58.png"
  username="test3">
    <stats/>
  </user>
</users>
```

Anexo 8 - Demonstração de Login:

```
=====
                        LOGIN
=====
Type 'exit' to go back to the main menu.

Username: rui

Password: 123
Login successful! Welcome back, rui!

≡ MainMenuScreen ≡
1. Play Dots and Boxes!
2. View profile
3. Search for another player
4. Logout
5. Exit

rui>
```

Anexo 9 - Demonstração de ver perfil:

```
≡ Your Profile ≡
User ID: 6151575b-8ecf-4eac-b760-fd20a3d2c8d0
Username: rui
nationality:
Date of Birth:
stats:
  Games played: 0
  Games won: 0
  Games lost: 0
  Win-rate: 0.0
  Total Play Time: 0.0 minutes

1. Edit Profile
2. Back to main menu

>> |
```


Anexo 10 - Demonstração de procurar por Players:

```
Type to search for new players ("back" to go to menu):

>> test

≡≡ SearchResultsScreen ≡≡
1. back
2. test
3. test2
4. test3

>> 2
```

Anexo 11 - Demonstração do ecrã de jogo e de realizar uma jogada:

```
=====
test: 0 | rui: 0
=====
  0   1   2   3   4 (X)
0 *---* * * *
  |
1 *---* * * *
2 * * * * *
3 * * * * *
4 * * * * *
(Y)

Your turn! Enter coordinates (x1 y1 x2 y2) or 'surrender': 0 1 0 0
```

Anexo 12 - Demonstração da da opção de desistir:

```
=====
test: 1 | rui: 0
=====
  0  1  2  3  4  (X)
0 *---* * * *
  | t |
1 *---* * * *

2 * * * * *
3 * * * * *
4 * * * * *
(Y)

Your turn! Enter coordinates (x1 y1 x2 y2) or 'surrender':
Game> surrender
Surrendering...
```